The Islamia University of Bahawalpur
Department of Information Technology

INFT-4041 · BSIT · 5th Semester · Fall 2026

E-Commerce Application Development

Chapter

16

Securing the Shop

Part IV — Money and Identity

Taught in Laboratory	Week 12
	The OWASP checks against your own container, before and after hardening
Assessment	**Quiz 2 (Ch 10–15)**

This is a class extract. It is *Chapter 16* of the unified handout, printed on its own so that a week's reading is a week's reading. Page numbers restart at 1 and are local to this extract; chapter, figure and table numbers are the ones the full volume gives them, so a reference you follow here lands in the same place there.

Cross-references to other chapters print as their numbers in the full volume. For the whole course in one file — front matter, the HEC tracing table, all twenty-three chapters and all seven appendices — see `ECAD_Fall2026_Handout.pdf`.

Dr. Mustafa Hameed · Department of Information Technology

Generated from `parts/` by `makechapters.py`; do not edit this PDF's source by hand

Chapter 16

Securing the Shop

Learning Outcomes

By the end of this chapter you will be able to:

- **State** the legal boundary of security testing, and **confine** your own work inside it.
- **Measure** a shop's security posture as a set of yes/no questions, and **repeat** the measurement after a change.
- **Distinguish** faults the framework already prevents from faults the deployment introduces, with evidence for each.
- **Apply** a hardening configuration, and **quantify** what it closed and what it did not.
- **Judge** which remaining findings need a decision rather than a setting, and **argue** for one.

Before You Start

Chapter 5: the container, Apache, and where configuration lives. Chapter 7: the upload directory and what is served from it. Chapter 15: passwords, lockout, and that a defence can create a leak. **Appendix A's scope statement, which you must read before the lab.**

Key Terms in This Chapter

threat model • attack surface • security header • content security policy • clickjacking • MIME sniffing • CORS • information disclosure • hardening • defence in depth • SQL injection • cross-site scripting • CSRF token • PECA 2016

16.1 Where You May Point This Chapter

Read this before the laboratory. It is not a formality.

Every exercise in this chapter runs against **the container on your own machine**, reached at `localhost`, which you installed, which contains no real person's data, and which you can destroy and rebuild with one command.

Running the same checks against any system you do not own and have not been authorised in writing to test is a criminal offence in Pakistan under the **Prevention of Electronic Crimes Act 2016**, and under the equivalent law of almost every other country. Sections 3 and 4 of that Act cover unauthorised access to an information system and to data, with imprisonment and fine attached to each.

There is no educational exemption. "I was learning", "I did no harm", "the site was insecure" and "I told them afterwards" are not defences, and a university assignment is not authorisation. The authorisation that counts is a written scope agreement from the system's owner, naming what may be tested and when.

What this means in practice: the only address that appears in this chapter is `localhost:8090`. If you find yourself typing anything else, stop.

16.2 Measure First, and Measure the Right Thing

The temptation in a security chapter is to start with the famous attacks. It is worth resisting, because the famous attacks are the ones the framework already handles, and starting there teaches you to look in the wrong place.

Measured: The two attacks everyone expects, against your own search box

From `code/m16_security.py -injection`. Both probes are harmless: one is a quotation mark, the other is a script tag that would open an alert box in your own browser.

Probe	Bytes	Verbatim?	Escaped?
a normal search	38,406	yes	yes
one quotation mark	26,440	no	no
`ipod' OR '1'='1`	26,508	yes	yes
`<script>alert(1)</script>`	26,648	**no**	yes
`<img src=x onerror=...>`	26,626	**no**	yes

The SQL probe returns a *shorter* page than the plain search — 26,508 bytes against 38,406. It found nothing, because the whole string including the quotation marks was searched for literally. The query's meaning did not change, which is what a parameterised or properly escaped query means.

The script probes never appear as markup, only in their escaped form. Twig auto-escapes everything it prints unless explicitly told not to, so the tag becomes text.

So both classic attacks are already handled. That is the useful finding, and it redirects the rest of the chapter: whatever is wrong with this shop is not in its application code.

16.3 What Is Actually Wrong, and It Is All Deployment

Measured: Fifteen questions, asked of the shop as installed

From `code/m16_security.py`. Each is a yes/no question you could check by hand with `curl`. "Pass" always means the safe answer.
Part one: what the shop says about itself in every response.

Question	Before	After	Cost to fix
Server header hides the version	fail	pass	1 line
`X-Powered-By` absent	fail	pass	1 line
`X-Content-Type-Options: nosniff`	fail	pass	1 line
`X-Frame-Options` or frame-ancestors	fail	pass	1 line
Content-Security-Policy set	fail	pass	1 block
`Referrer-Policy` set	fail	pass	1 line
`Permissions-Policy` set	fail	pass	1 line
No wildcard CORS with credentials	fail	pass	2 lines

Part two: what the shop will hand over if you ask for it.

Question	Before	After	Cost to fix
Error log not readable over HTTP	**fail**	**pass**	1 block
Session cookie is HttpOnly	pass	pass	—
Session cookie has SameSite	pass	pass	—
`system/storage/` not browsable	pass	pass	—
`install/` removed	pass	pass	—
`config.php` not served as text	pass	pass	—
Admin not at the default path	fail	**fail**	a decision
Both parts together	**5 of 15**	**14 of 15**	

One configuration file closed nine findings. Not one of them required a change to the shop's code, and not one of them took more than a few lines.

The serious one is the error log. Before hardening, anybody at all who asked the shop for `system/storage/logs/error.log` was given it — verified by writing a marker string into the file and then fetching it back over HTTP. A PHP error log contains file paths, SQL fragments, and often the contents of whatever request failed.

⚠ Why the shipped protection does not work, in two parts

The platform knows about this. It ships a file called `.htaccess.txt` containing, among other things, exactly the rule that would have blocked the log. Every guide to securing this software says: rename it.

First, nobody renames it. The installer does not, the Docker image does not, and a fresh install therefore has no `.htaccess` at all.

Second, and worse: renaming it would not help here. Debian's `apache2.conf` sets `AllowOverride None` for the document root, so Apache never reads `.htaccess` files. You could rename it, confirm the file is in place, and have changed nothing — with no error, no warning, and no way to tell.

This is why the fix in `code/shop/security.conf` is a server configuration rather than a `.htaccess`. It is also why the advice you will find online for this platform is worse than useless: it is confidently wrong, it has been repeated for fifteen years, and following it produces a shop that is exactly as exposed as before while its owner believes otherwise.

Verify that a control works. Do not verify that you installed it.

16.4 Three Mistakes Made While Writing This Chapter

The hardening file did not work the first time, or the second. The failures are kept because each one is a way a real control silently does nothing.

</> What went wrong, and how the measurement found it

1. **`Header always unset` removed nothing.** The CORS headers were still there after hardening. `mod_headers` keeps two tables: `always` applies to every response including errors, and the default applies only to successful ones. A header PHP sets on a 200 lives in the second table. **Both forms are needed**, and using only the one whose name sounds more thorough removes nothing.

2. **`ServerTokens` was ignored after a reload.** It is read only when Apache *starts*. A graceful reload applies every other directive in the file and silently skips this one.
3. **The file was overridden by Debian's own.** Apache reads `conf-enabled` in filename order, and the last setting of a server-level directive wins. A file named `security-ecad.conf` sorts *before* `security.conf` — because `-` precedes `.` — so Debian's `ServerTokens OS` was applied after it. Renaming the file to `zz-security-ecad.conf` fixed it.

All three produced a valid configuration that Apache accepted without complaint, and in all three the header was simply absent. A security control that fails open and fails quietly is the only kind worth being afraid of, and the only defence is to measure the result rather than the action. Each of these took one run of the harness to find and would have survived any amount of re-reading the file.

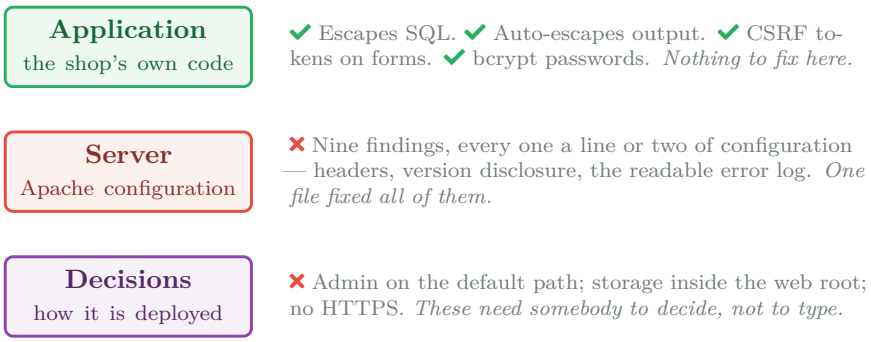


Figure 16.1: Where a stock shop's security problems actually live. The layer students expect to fix is the one that was already correct.

16.5 What a Header Cannot Fix

One finding survived the configuration file, and it is the honest one.

The three that need a decision

1. **The admin panel is at `/admin/`.** Renaming the directory and the matching path in `admin/config.php` is not security — anyone who looks will find it — but it removes the shop from every automated scan that tries one path and moves on. It costs a configuration change and a note for whoever maintains the shop next, which is why it is a decision rather than a setting.
2. **`system/storage/` is inside the web root.** Blocking it in the server config, as this chapter did, is the fix you can apply today. **Moving it outside the web root is the fix that is still correct after somebody edits the server config in two years.** Prefer the control that does not depend on remembering.
3. **There is no HTTPS.** On `localhost` it does not matter. On a real shop everything in this book — the session cookie, the password, the order — travels in clear text without it, and `Strict-Transport-Security` is deliberately left out of the hardening file because switching it on without a certificate makes the shop unreachable in your own browser for months. Chapter 20 turns it on where there is something to turn it on for.

The defaults this course has been using, and must not keep

The shop was installed with `admin`/`admin123`, a database user `shop`/`shoppw`, and a root password of `rootpw`. Those are in the handout, in the repository, and in this paragraph.

They are acceptable for exactly one thing: a container on your own machine, with no data in it, not reachable from your network. They are documented as teaching defaults rather than hidden, because a weak password nobody mentions is worse than a weak password everybody knows about.

Any shop that leaves this machine changes all three before it does, along with the minimum password length of six characters noted in Chapter 15. Chapter 20 makes that a checklist item with nothing optional about it.

Common Pitfalls

- **Testing anything but your own container.** See the first page of this chapter, and Appendix A.
- **Starting with SQL injection and cross-site scripting.** Both were already handled; the time was better spent elsewhere.
- **Renaming `.htaccess.txt` and believing you are done.** `AllowOverride None` means the file is never read.
- **Verifying that you applied a control rather than that it works.** Three separate controls in this chapter were applied correctly and did nothing.
- **Using `Header always unset` alone** to remove a header from a successful response.
- **Reloading instead of restarting** after changing a directive that is only read at startup.
- **Setting `Strict-Transport-Security` without a certificate**, which locks you out of your own shop.
- **Writing a Content-Security-Policy that breaks the theme**, then removing it entirely instead of narrowing it.
- **Treating a passing checklist as a secure shop.** Fourteen of fifteen is a measurement, not a guarantee.

Try It Yourself: Break into your own shop, then shut the door

```
# SCOPE: localhost only. Your container, your machine, no real data.
# Re-read the first page of this chapter before you begin.

# --- 1. Ask what it tells you, before changing anything -------------
cd code
curl -sI http://localhost:8090/ | head -20
# List every header that names a version or a product.

# --- 2. Read your own error log over HTTP ---------------------------
docker compose exec shop sh -c \
  'echo "SECRET-TEST-STRING" >> system/storage/logs/error.log'
curl -s http://localhost:8090/system/storage/logs/error.log
# If you see your string, so would anyone else.

# --- 3. Run the battery ---------------------------------------------
python m16_security.py --check

# --- 4. Try the two attacks you came here for -----------------------
python m16_security.py --injection
# Predict the result first. Most people predict at least one works.

# --- 5. Find out why the shipped protection does not ----------------
docker compose exec shop sh -c 'ls -la .htaccess*'
docker compose exec shop sh -c \
  'grep -n AllowOverride /etc/apache2/apache2.conf'
# Two separate reasons. Write both down.
```

```
# --- 6. Harden, and measure the difference --------------------------
python m16_security.py
# Read shop/security.conf first and predict the score.

# --- 7. Check the shop still works ----------------------------------
python waitfor.py
python m10_checkout.py --repeat 1
python m10_checkout.py --clean
# A hardening change that breaks checkout is not a hardening change.

# --- 8. Close the last finding --------------------------------------
# Rename the admin directory AND the path in admin/config.php. Then
# re-run the battery. Say why this is weaker than the other fixes.

# --- 9. Write the five lines you would hand to a client -------------
# What you changed, what is still open, what each one would cost,
# and which one you would do first.
```

What to notice. Step 4 against step 3. The attacks you have heard of were already stopped; the shop was giving away its error log. Security attention goes where the stories are, and the stories are about the wrong layer.

Checkpoint — Test Yourself

1. A colleague renames `.htaccess.txt` to `.htaccess` on this stack and reports the shop secured. Give the two independent reasons that is wrong, say what you would run to demonstrate it in one command, and state the general lesson about verifying controls.
2. The battery went from 5 of 15 to 14 of 15 with one configuration file. Explain why none of the nine fixes touched the shop's code, what that says about where to look when you inherit a shop, and which single remaining finding you would address first and why.
3. Three controls in this chapter were written correctly, accepted by Apache without complaint, and did nothing. Name the mechanism behind any two, and describe a working habit that would catch all three without knowing any of them in advance.

Chapter Summary

- Every exercise is confined to your own container on `localhost`. Testing anything else is a criminal offence under PECA 2016, with no educational exemption.
- Measured: SQL injection and cross-site scripting against the search box were both already handled — the injection string was searched for literally and returned fewer results, and the script tag appeared only escaped. The famous attacks were not the problem.
- Measured: the shop as installed passed 5 of 15 checks. After one configuration file it passed 14. Nothing in the shop's code changed.
- The serious finding was the PHP error log, readable over HTTP — confirmed by writing a marker into it and fetching it back.
- The shipped protection fails twice over: `.htaccess.txt` is never renamed, and `AllowOverride None` means it would not be read if it were. The advice repeated everywhere online for this platform does nothing on a modern deployment.
- Three controls were applied correctly and silently did nothing: `Header always unset` on a successful response, `ServerTokens` after a reload rather than a restart, and a file that sorted before Debian's own. Verify the result, never the action.

- What a header cannot fix needs a decision: move the admin path, move storage out of the web root, and serve over HTTPS. Prefer the control that does not depend on somebody remembering.
- The default credentials used throughout this course are documented rather than hidden, and are acceptable for a local container and nothing else.

Review Questions

1. **(MCQ)** The stock shop passed how many of the fifteen checks? (a) 0; (b) 5; (c) 11; (d) 14.
2. **(MCQ)** Renaming `.htaccess.txt` to `.htaccess` on this stack achieves: (a) full protection; (b) partial protection; (c) nothing, because Apache does not read it; (d) nothing, because the file is empty.
3. **(Short)** Explain what `X-Content-Type-Options: nosniff` prevents, and connect it to Chapter 7's upload findings.
4. **(Short)** Why is `Strict-Transport-Security` deliberately absent from this chapter's hardening file?
5. **(Short)** Give one reason moving `system/storage/` out of the web root is better than blocking it in the server configuration, even though both work today.
6. **(Applied)** You inherit a shop you did not build. Write the order in which you would investigate it, with the specific command or observation at each step, and say at which point you would stop and report rather than continue — and why that point exists.
7. **(Applied)** Design a Content-Security-Policy for a shop whose theme uses inline scripts, a payment provider's hosted field, and an analytics service. State each directive, what it allows, what it blocks, how you would roll it out without breaking the shop, and how you would know it was working.